STOCK MARKET DATA PIPELINE

REVIEW – 4

Batch 17 - Section 4

Team members

Roll No	Name
2420030415	G.Rithwik
2420030592	J.Kumudavalli
2420090113	T.Ram Tejaswi Reddy

Under the guidance of

Dr. Nalla Shirisha

Associate Professor

Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Table of contents

- ABSTRACT
- INTRODUCTION
- EXISTING SYSTEM
- PROPOSED SYSTEM
- OBJECTIVES
- LITERATURE SURVEY
- SOFTWARE REQUIREMENTS
- HARDWARE REQUIREMENTS
- FUNCTIONAL REQUIREMENTS
- NON-FUNCTIONAL REQUIREMENTS
- SYSTEM ARCHITECTURE
- UML DIAGRAM
- METHODOLOGY, SYSTEM DESIGN AND ARCHITECTURE
- OUTPUT AND RESULTS

Abstract

The Stock Market Data Pipeline is designed to collect, process, and analyze stock market data in a structured and efficient manner. Stock market data such as opening price, closing price, highest price, lowest price, trading volume, and timestamps is collected from financial data sources or APIs. The pipeline cleans and preprocesses the collected data by handling missing, duplicate, inconsistent, or incorrect records and then transforms it into a suitable format for analysis. The processed data can be stored and analyzed to identify stock price movements, trends, trading patterns, and market behavior. Visualization techniques can be used to present the analyzed information through charts or dashboards, helping users understand market trends and make data-driven financial decisions.

Introduction

The stock market produces a large amount of data every day through buying and selling activities. This data includes open, high, low, close prices, trading volume, timestamps, and other market information.

Analyzing raw stock market data directly can be difficult because the data may contain missing values, duplicate records, incorrect formats, and inconsistencies.

A Stock Market Data Pipeline solves this problem by collecting market data and passing it through different processing stages. The pipeline cleans and transforms the raw data before storing it in a structured format.

The processed data can then be analyzed to identify market trends, price movements, volatility, and trading patterns.

Therefore, a stock market data pipeline helps make financial data organized, accessible, and easier to analyze.

LIMITATIONS OF EXISTING SYSTEM

- **Manual Data Collection** – Stock data is often collected manually from different sources.
- **Poor Data Quality** – Missing, duplicate, inconsistent, or incorrect records can remain.
- **Time-Consuming Processing** – Cleaning and analyzing large amounts of data manually takes more time.
- **Limited Real-Time Monitoring** – Traditional systems may not continuously process live market data.
- **Difficult Data Management** – Data from multiple sources can be difficult to organize and maintain.
- **Slow Trend Analysis** – Identifying price movements and trading patterns takes more effort.
- **Limited Visualization** – Data may be presented as raw tables instead of interactive charts or dashboards.
- **Higher Risk of Human Errors** – Manual calculations and data entry can lead to inaccurate results.

PROPOSED SYSTEM

The proposed system is an **automated stock market data pipeline** that converts raw market data into meaningful insight through multiple processing stages.

Pipeline:

Stock Market API / Historical Dataset

↓

Data Collection – Collect stock symbol, date, Open, High, Low, Close and Volume

↓

Data Cleaning – Handle missing, duplicate and inconsistent records

↓

Data Transformation – Calculate price changes, returns, moving averages and volume patterns

↓

Data Storage – Store processed data in SQL/database or file system

↓

Data Analysis – Analyze prices, volume, trends and market patterns

↓

Visualization – Generate charts and dashboards using Matplotlib/Plotly

↓

Market Insights – Provide understandable information about market movements

Technologies Used

- **Python** – Pipeline development and processing
- **Pandas** – Cleaning and transformation
- **NumPy** – Numerical calculations
- **SQL/Database** – Data storage
- **Matplotlib/Plotly** – Visualization
- **Jupyter Notebook** – Development and testing
- **Stock Market API** – Historical/real-time data collection

Main advantage: It reduces manual effort, improves data quality, speeds up analysis, and makes market trends easier to understand.

Objectives

- To collect historical and real-time stock market data from reliable data sources or APIs.
- To clean and preprocess the collected data by handling missing, duplicate, and inconsistent records.
- To transform raw stock market data into a structured format suitable for analysis.
- To analyze stock prices, trading volume, market trends, and price movements.
- To identify useful patterns and trends from historical stock market data.
- To visualize stock market information using charts and dashboards.
- To support efficient and data-driven financial market analysis.

Literature Survey

S.No	Research Paper	Authors	Link	Relevance / Finding
1	Cloud based evaluation of databases for stock market data	Baldeep Singh, Randall Martyr, Thomas Medland, Jamie Astin, Gordon Hunter, Jean-Christophe Nebel	https://link.springer.com/article/10.1186/s13677-022-00323-4	Evaluates cloud-based databases for handling and processing stock-market data.
2	Deep learning in the stock market—a systematic survey of practice, backtesting, and applications	Systematic survey paper	https://link.springer.com/article/10.1007/s10462-022-10226-0	Discusses market data, APIs/data feeds and the importance of processing financial data for stock-market analysis.
3	Machine learning in stock market forecasting: a comprehensive review	Moksh Khemka, Kamal Haddad, Tanzim Redwan, Adib Ahmed et al.	https://link.springer.com/article/10.1007/s10791-026-10491-5	Reviews machine-learning approaches for stock-market forecasting and highlights the importance of suitable financial data.
4	Scalable Real-Time Market Data Processing Architecture for High-Volume Multi-Asset Analytics in Fund Management	Phaneendra Vayu Kumar Yerra	https://mail.ijisae.org/index.php/IJISAE/article/view/7973	Studies scalable architectures for high-volume, real-time financial market-data processing, including streaming and pipeline architectures.
5	Scaling a Data Warehouse System with Machine Learning Integration for Stock Market Forecasting	Keona Gagnier, Ajitesh Parihar, Joshua Padron-Uy, Gaëtan Hains, Albert Wong	https://d197for5662m48.cloudfront.net/documents/publicationstatus/247212/preprint_pdf/66aee7a58b67a5b0a5266298b0a0fbe8.pdf	Directly relevant to your project because it investigates an ETL-based data pipeline + data warehouse + stock-market forecasting system.

Research Gap	Existing studies focus mainly on databases, forecasting, or high-volume processing individually. Our project combines data ingestion, cleaning, transformation, storage, analysis and visualization into one complete stock-market data pipeline.

SOFTWARE REQUIREMENTS

- Operating System: Windows / Linux
- Programming Language: Python
- Libraries: Pandas, NumPy
- Database: SQL
- Visualization: Matplotlib / Plotly
- Development Environment: Jupyter Notebook
- Data Source: Stock Market API / Historical Dataset
- Dashboard: Python-based visualization/dashboard

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or equivalent
- RAM: 4 GB minimum
- Storage: 10 GB available space
- System: Desktop/Laptop
- Internet: Required for Stock Market API data collection
- Display: Standard monitor

Processor: Core i3 | RAM: 4 GB | Storage: 10 GB | Internet: Required | Device: Laptop/Desktop

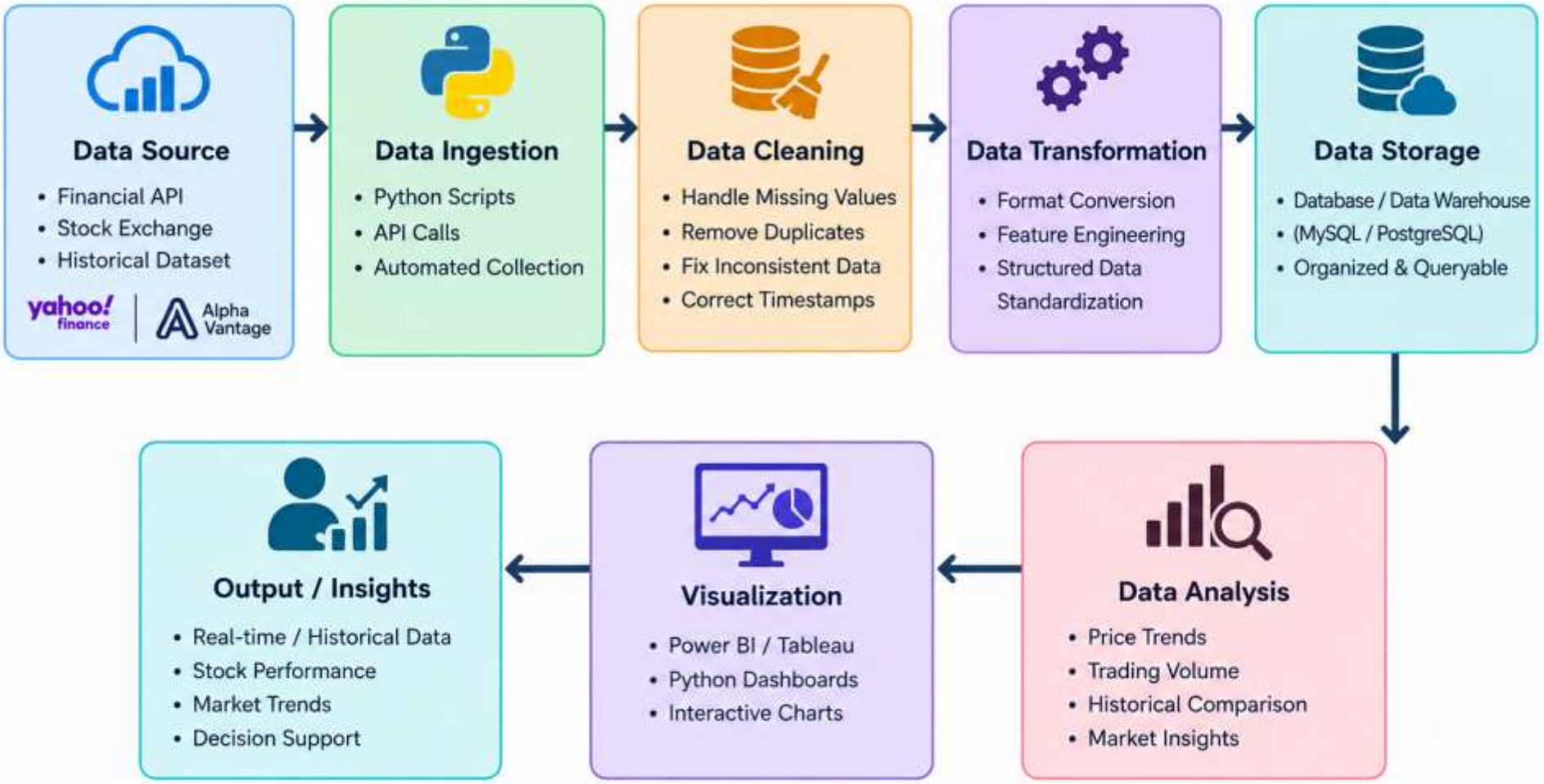
FUNCTIONAL REQUIREMENTS

- **Data Collection** – Collect historical and real-time stock market data from APIs or datasets.
- **Data Cleaning** – Handle missing, duplicate, inconsistent, and incorrect records.
- **Data Transformation** – Convert raw data into a structured format and calculate useful indicators.
- **Data Storage** – Store processed stock data in a database or file system.
- **Data Analysis** – Analyze stock prices, trading volume, trends, and market patterns.
- **Data Visualization** – Generate charts and dashboards to display market information.
- **Market Insights** – Provide meaningful insights about stock movements and trends.
- **Real-Time Monitoring** – Support continuous monitoring when connected to a live data source.

NON FUNCTIONAL REQUIREMENTS

- Performance – Process large amounts of stock data efficiently.
- Scalability – Handle increasing amounts of market data.
- Reliability – Provide consistent and accurate processing.
- Security – Protect stored and processed financial data.
- Maintainability – Keep the pipeline modular and easy to update.
- Usability – Present results through clear charts and dashboards.
- Availability – Support continuous data processing when live data is available.
- Data Quality – Maintain clean, consistent, and accurate data.

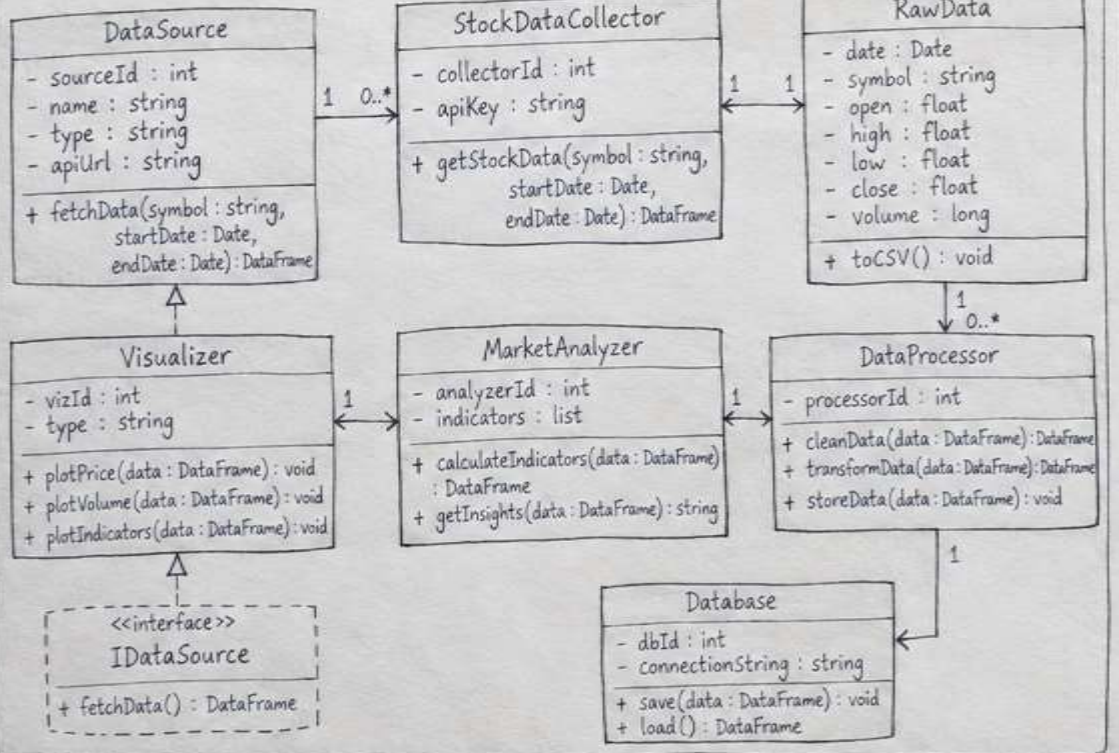
Stock Market Data Pipeline – System Architecture



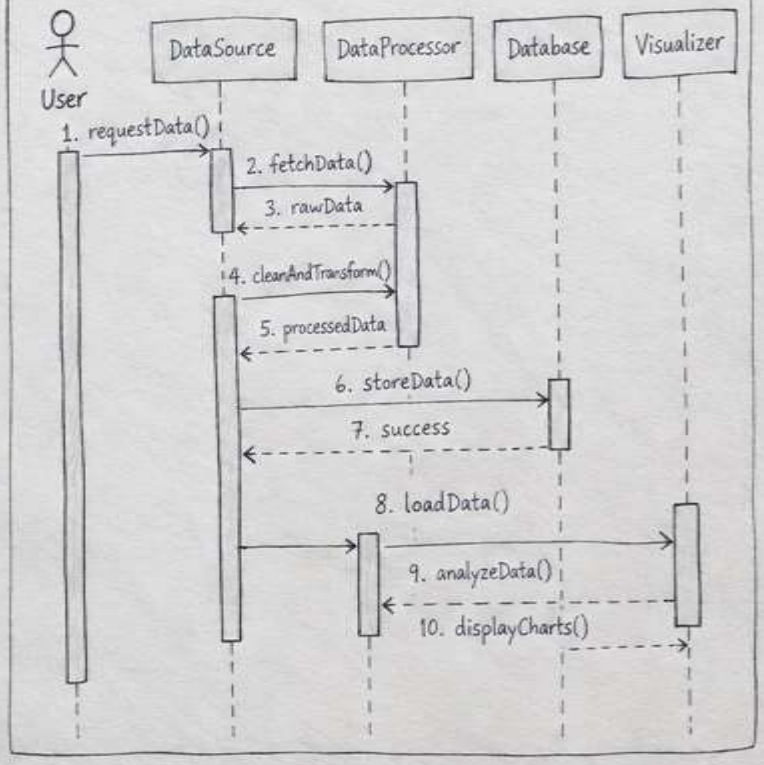
Stock Market Data Pipeline - UML Design

System Design Using UML Diagrams

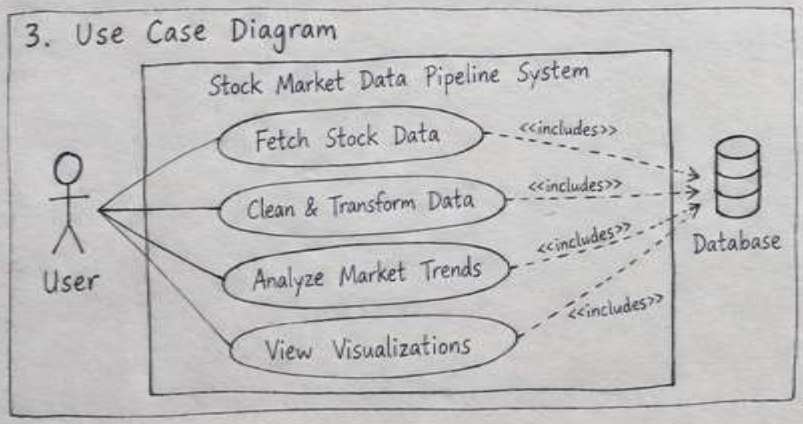
1. Class Diagram



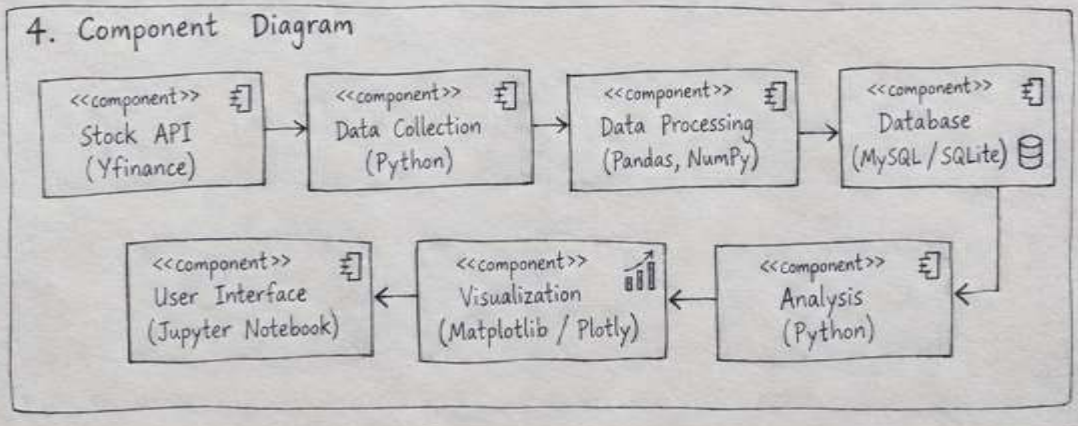
2. Sequence Diagram



3. Use Case Diagram



4. Component Diagram

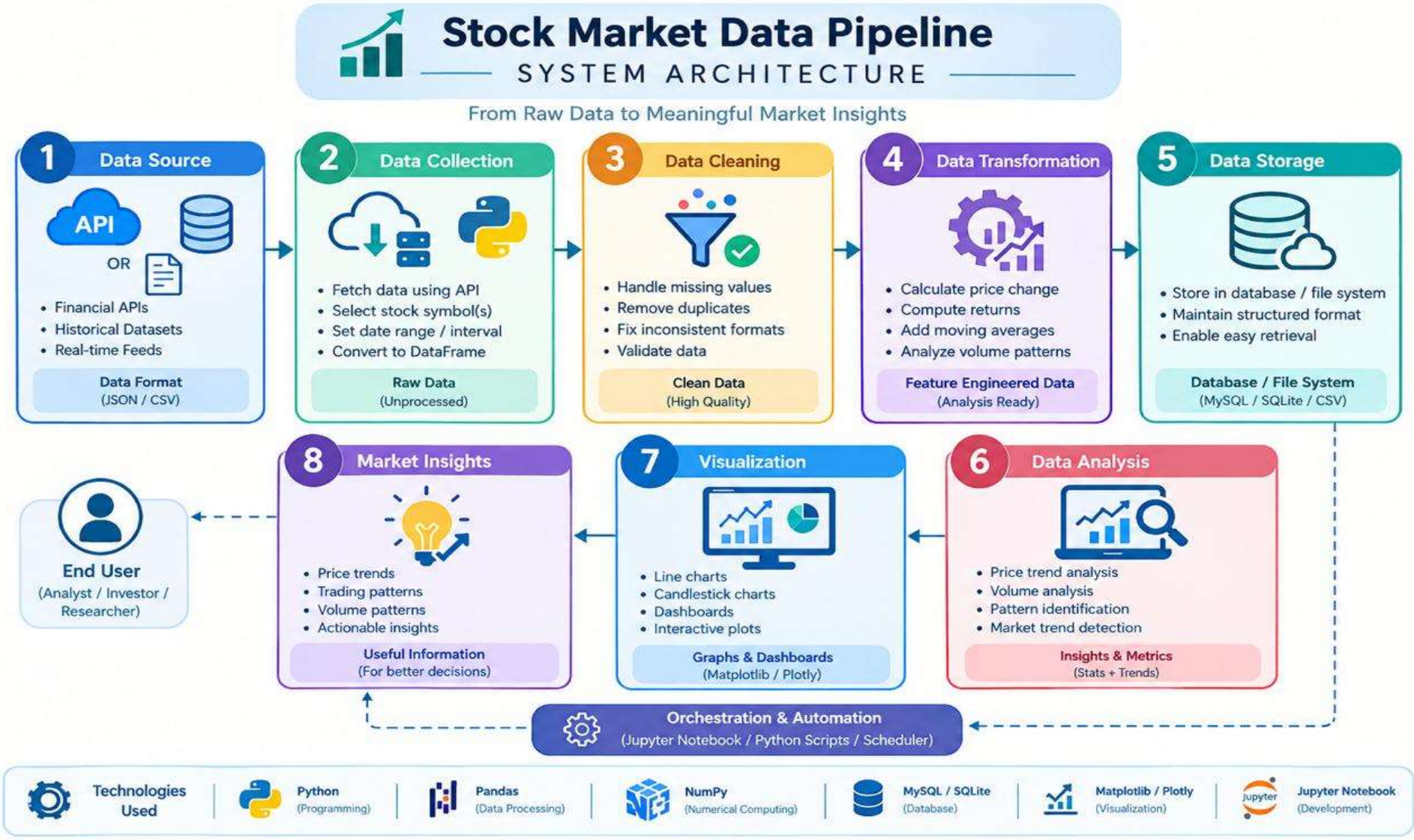


Methodology, System Design and Architecture

The Stock Market Data Pipeline begins by collecting stock market data from reliable data sources such as financial APIs or historical datasets. The collected data generally contains information such as stock symbol, date, opening price, highest price, lowest price, closing price, and trading volume. This raw data is then passed through preprocessing, where missing, duplicate, and inconsistent records are identified and handled. The data is converted into a consistent and structured format so that it can be used effectively for further analysis.

After preprocessing, the cleaned data is transformed into a suitable format for analysis. Different features such as price changes, returns, moving averages, and trading-volume patterns can be derived from the available data. The transformed data is then stored in a structured database or file system for efficient access. Finally, the processed stock market data is analyzed to identify price movements, trends, and market patterns, and the results are presented through visualizations and dashboards.

System Architecture



Use of Modern Tools and Technologies

- **Technologies Used**
- **Python**
  Used for data collection, processing, analysis, and pipeline development.
- **Pandas**
  Used for data cleaning, manipulation, transformation, and analysis.
- **NumPy**
  Used for numerical operations and financial calculations.
- **SQL / Database**
  Used to store and retrieve processed stock market data.
- **Matplotlib / Plotly**
  Used to visualize stock prices, trading volume, and market trends.
- **Jupyter Notebook**
  Used for development, experimentation, and testing.
- **Stock Market API**
  Used to obtain historical or real-time market data.

Benefits

Automated Data Processing – Reduces manual effort in collecting and processing stock market data.

Improved Data Quality – Handles missing, duplicate, and inconsistent data.

Faster Analysis – Processes large amounts of stock market data efficiently.

Better Trend Identification – Helps identify price movements, market trends, and trading patterns.

Easy Visualization – Presents stock information through charts and dashboards.

Real-Time Monitoring – Can support continuous monitoring of market data when connected to live data sources.

Data-Driven Decisions – Provides useful insights for financial and market analysis**.**

Implementation, Coding Quality and Functionality

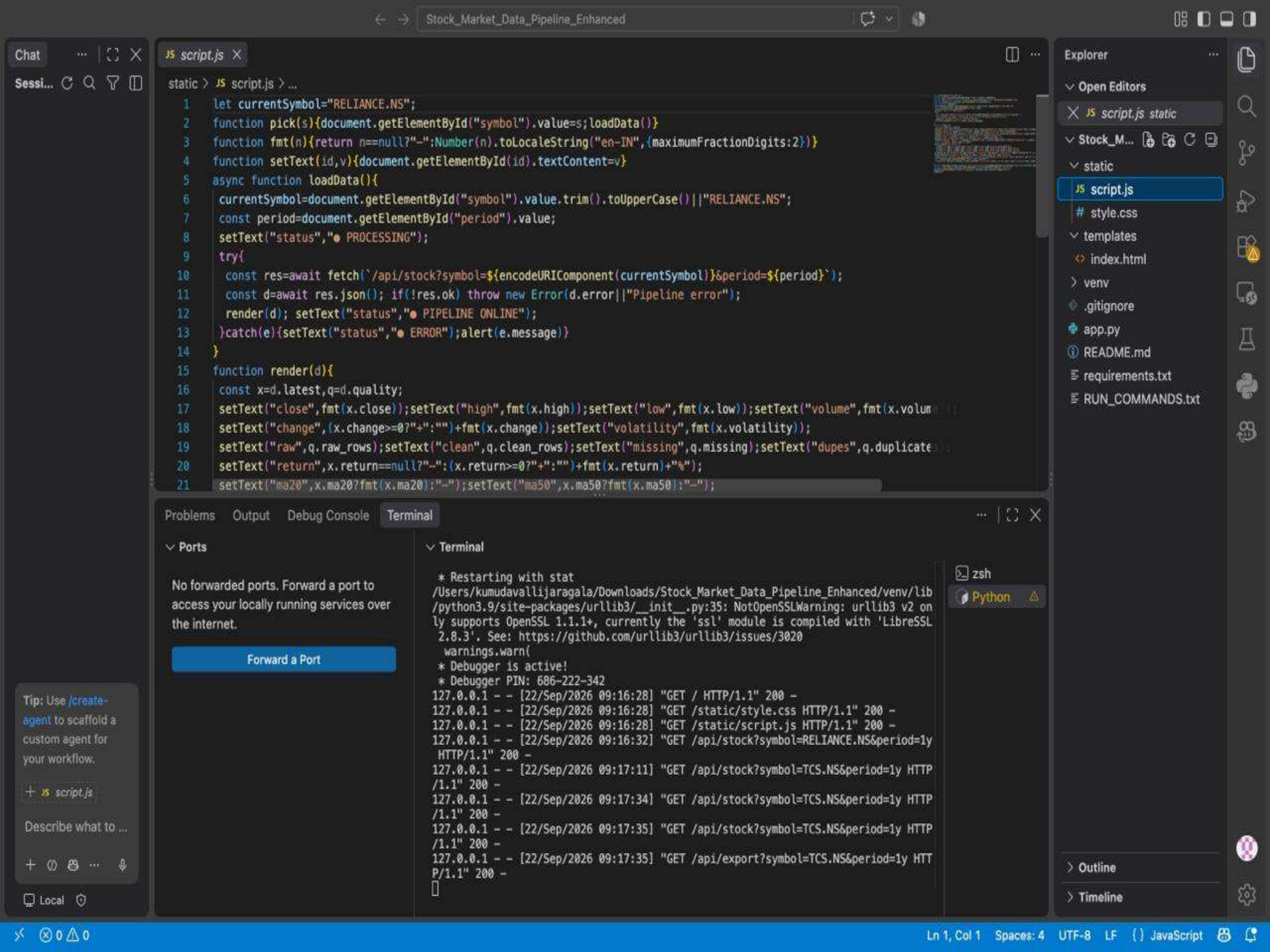
- **Implementation**
- Develop the stock market data pipeline.
- Collect stock market data from an API/data source.
- Store raw market data.
- Clean missing and inconsistent records.
- Transform the collected data.
- Calculate useful market indicators.
- Analyze stock price and volume trends.
- Generate charts and visualizations.
- Display processed information through a dashboard.

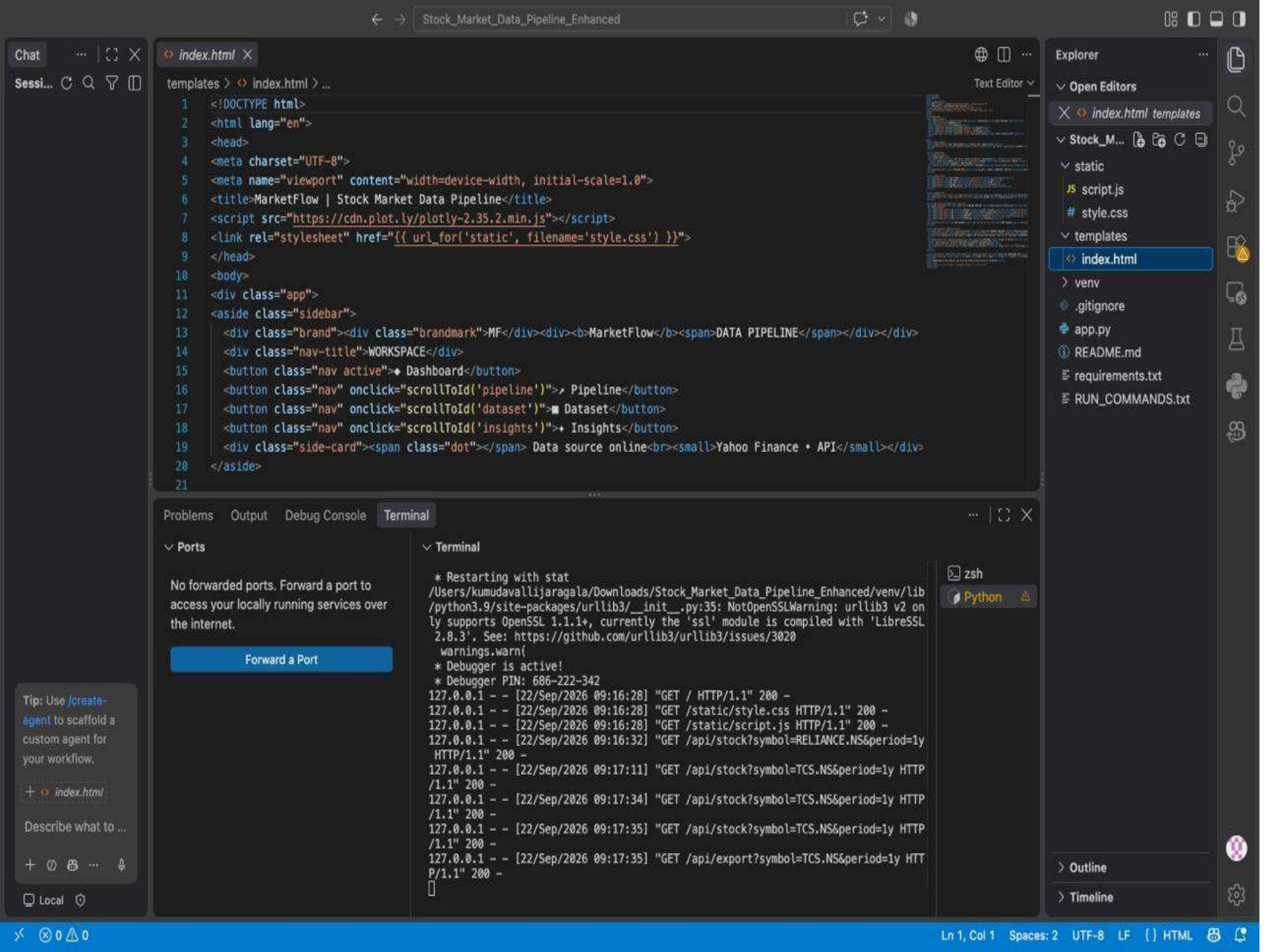
Coding Quality

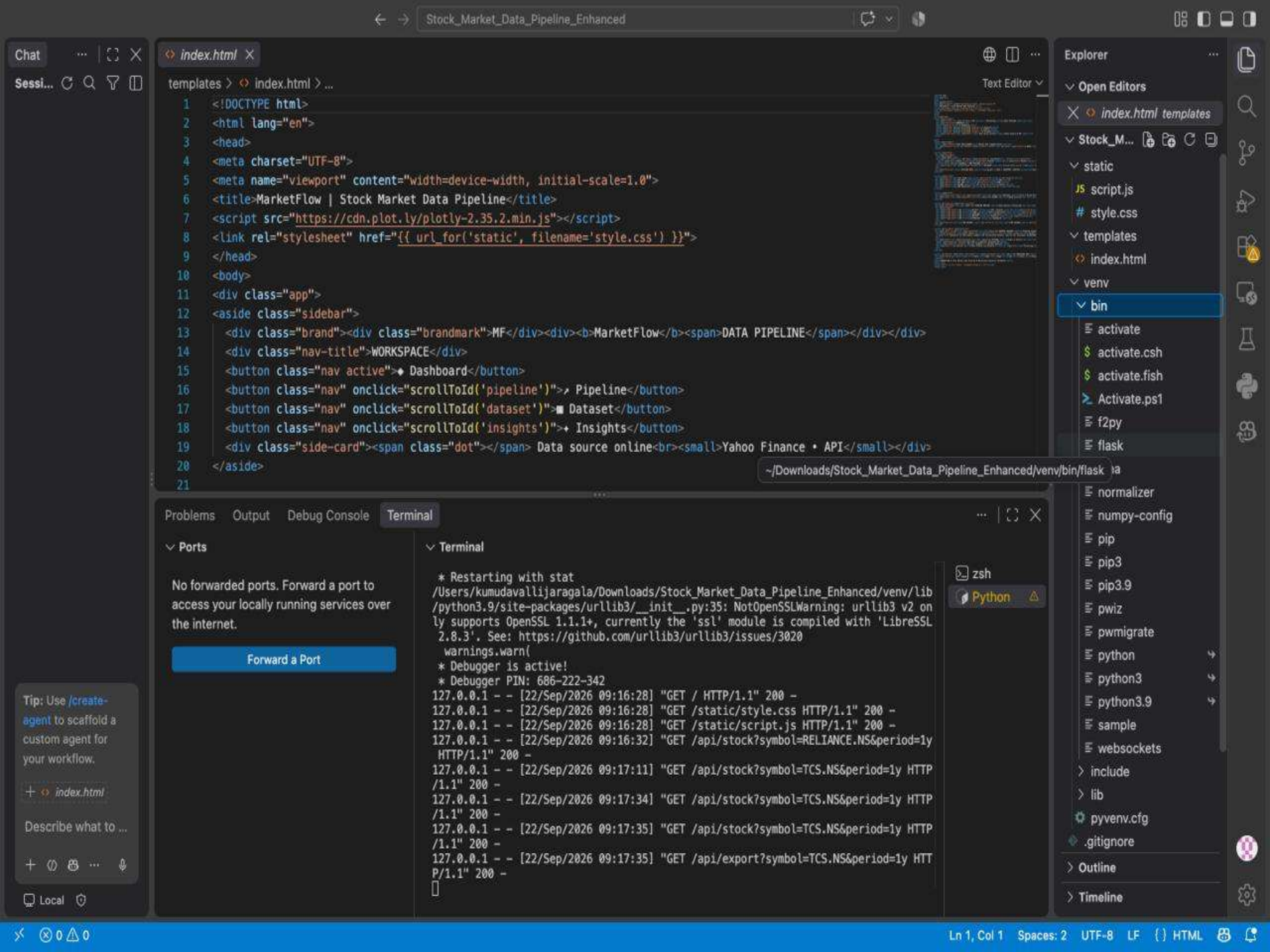
- Simple and structured implementation.
- Modular pipeline components.
- Clear and understandable Python code.
- Proper data preprocessing.
- Efficient data handling.
- Error handling for missing or invalid data.
- Reusable functions for different stocks.
- Maintainable and scalable implementation.

Functionality

The Stock Market Data Pipeline collects and processes financial market data and converts raw information into a structured dataset suitable for analysis. It performs data cleaning and transformation before storing the processed information. The system can analyze stock prices and trading volumes to identify important trends and patterns. The final results are presented using visualizations or a dashboard, allowing users to easily understand stock market movements and obtain useful insights.

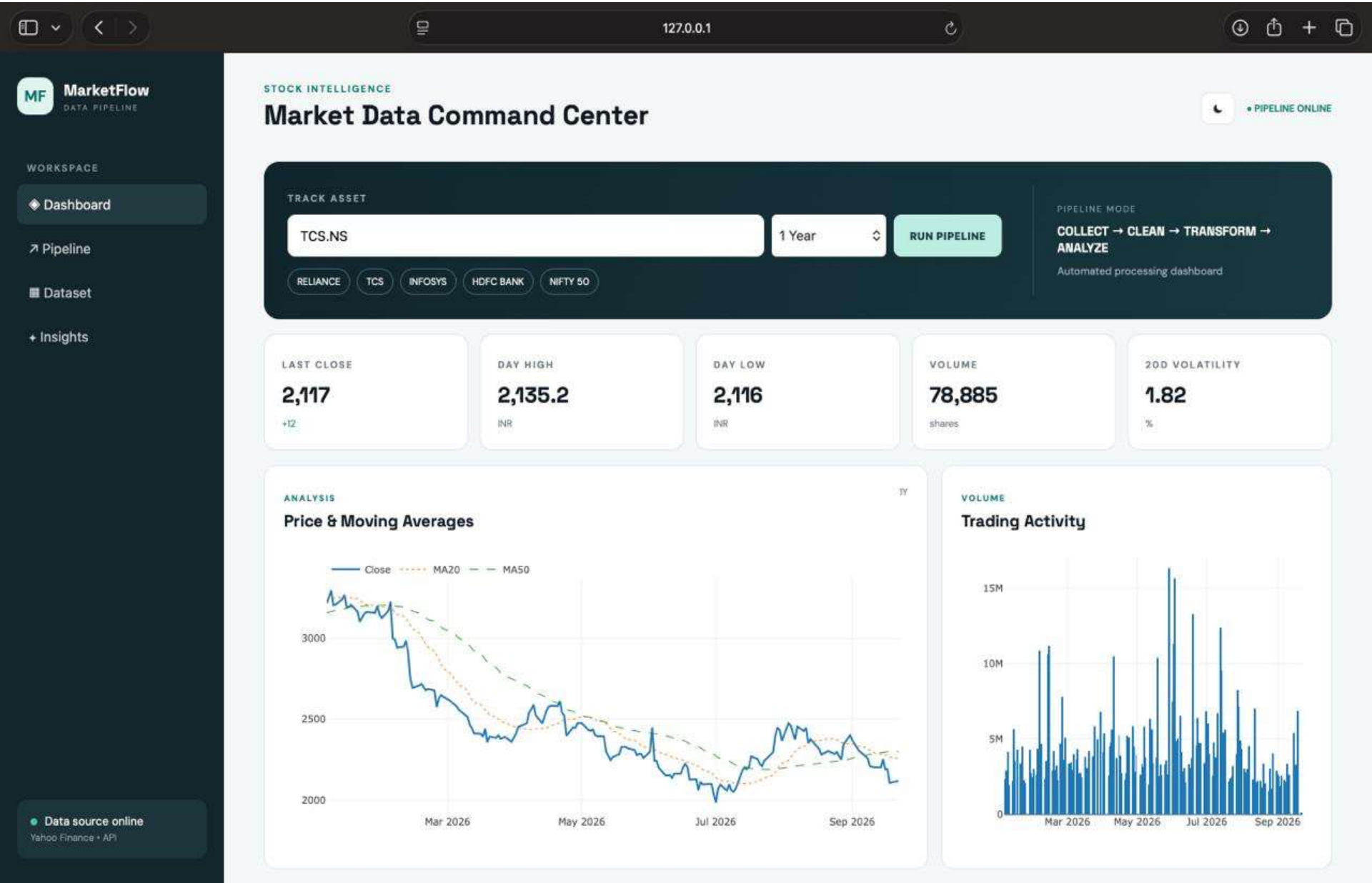

Stock_Market_Data_Pipeline_Enhanced
Chat
Sessi...
script.js
static > script.js > ...
let currentSymbol="RELIANCE.NS";
function pick(s){document.getElementById("symbol").value=s;loadData()}
function fmt(n){return n==null?"–":Number(n).toLocaleString("en-IN",{maximumFractionDigits:2})}
function setText(id,v){document.getElementById(id).textContent=v}
async function loadData(){
currentSymbol=document.getElementById("symbol").value.trim().toUpperCase()||"RELIANCE.NS";
const period=document.getElementById("period").value;
setText("status","● PROCESSING");
try{
const res=await fetch(`/api/stock?symbol=${encodeURIComponent(currentSymbol)}&period=${period}`);
const d=await res.json(); if(!res.ok) throw new Error(d.error||"Pipeline error");
render(d); setText("status","● PIPELINE ONLINE");
}catch(e){setText("status","● ERROR");alert(e.message)}
}
function render(d){
const x=d.latest,q=d.quality;
setText("close",fmt(x.close));setText("high",fmt(x.high));setText("low",fmt(x.low));setText("volume",fmt(x.volum
setText("change",(x.change>=0?"+":"")+fmt(x.change));setText("volatility",fmt(x.volatility));
setText("raw",q.raw_rows);setText("clean",q.clean_rows);setText("missing",q.missing);setText("dupes",q.duplicate
setText("return",x.return==null?"–":(x.return>=0?"+":"")+fmt(x.return)+"%");
setText("ma20",x.ma20?fmt(x.ma20):"–");setText("ma50",x.ma50?fmt(x.ma50):"–");
Problems
Output
Debug Console
Terminal
Ports
No forwarded ports. Forward a port to access your locally running services over the internet.
Forward a Port
Terminal
* Restarting with stat
/Users/kumudavallijaragala/Downloads/Stock_Market_Data_Pipeline_Enhanced/venv/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
* Debugger is active!
* Debugger PIN: 686-222-342
127.0.0.1 - - [22/Sep/2026 09:16:28] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:16:28] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:16:28] "GET /static/script.js HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:16:32] "GET /api/stock?symbol=RELIANCE.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:11] "GET /api/stock?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:34] "GET /api/stock?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:35] "GET /api/stock?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:35] "GET /api/export?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
zsh
Python
Explorer
Open Editors
script.js static
Stock_M...
static
script.js
style.css
templates
index.html
venv
.gitignore
app.py
README.md
requirements.txt
RUN_COMMANDS.txt
Outline
Timeline
Tip: Use /create-agent to scaffold a custom agent for your workflow.
script.js
Describe what to ...
Local
Ln 1, Col 1
Spaces: 4
UTF-8
LF
JavaScript


Stock_Market_Data_Pipeline_Enhanced
Chat
Sessi...
index.html
templates > index.html > ...
Text Editor
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>MarketFlow | Stock Market Data Pipeline</title>
7 <script src="https://cdn.plot.ly/plotly-2.35.2.min.js"></script>
8 <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
9 </head>
10 <body>
11
12 <aside class="sidebar">
13 MF MarketFlowDATA PIPELINE
14 WORKSPACE
15 <button class="nav active">◆ Dashboard</button>
16 <button class="nav" onclick="scrollToId('pipeline')">↗ Pipeline</button>
17 <button class="nav" onclick="scrollToId('dataset')">■ Dataset</button>
18 <button class="nav" onclick="scrollToId('insights')">✦ Insights</button>
19 Data source online
<small>Yahoo Finance • API</small>
20 </aside>
21
Problems
Output
Debug Console
Terminal
Ports
No forwarded ports. Forward a port to access your locally running services over the internet.
Forward a Port
Terminal
* Restarting with stat
/Users/kumudavallijaragala/Downloads/Stock_Market_Data_Pipeline_Enhanced/venv/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
* Debugger is active!
* Debugger PIN: 686-222-342
127.0.0.1 - - [22/Sep/2026 09:16:28] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:16:28] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:16:28] "GET /static/script.js HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:16:32] "GET /api/stock?symbol=RELIANCE.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:11] "GET /api/stock?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:34] "GET /api/stock?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:35] "GET /api/stock?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:35] "GET /api/export?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
zsh
Python
Tip: Use /create-agent to scaffold a custom agent for your workflow.
index.html
Describe what to ...
Local
Explorer
Open Editors
index.html templates
Stock_M...
static
script.js
style.css
templates
index.html
venv
.gitignore
app.py
README.md
requirements.txt
RUN_COMMANDS.txt
Outline
Timeline
Ln 1, Col 1
Spaces: 2
UTF-8
LF
HTML


Stock_Market_Data_Pipeline_Enhanced
Chat
Sessi...
index.html
templates > index.html > ...
Text Editor
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>MarketFlow | Stock Market Data Pipeline</title>
<script src="https://cdn.plot.ly/plotly-2.35.2.min.js"></script>
<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>

<aside class="sidebar">
MF MarketFlowDATA PIPELINE
WORKSPACE
<button class="nav active">◆ Dashboard</button>
<button class="nav" onclick="scrollToId('pipeline')">↗ Pipeline</button>
<button class="nav" onclick="scrollToId('dataset')">■ Dataset</button>
<button class="nav" onclick="scrollToId('insights')">✦ Insights</button>
 Data source online
<small>Yahoo Finance • API</small>
</aside>
Problems
Output
Debug Console
Terminal
Ports
No forwarded ports. Forward a port to access your locally running services over the internet.
Forward a Port
Terminal
* Restarting with stat
/Users/kumudavallijaragala/Downloads/Stock_Market_Data_Pipeline_Enhanced/venv/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
* Debugger is active!
* Debugger PIN: 686-222-342
127.0.0.1 - - [22/Sep/2026 09:16:28] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:16:28] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:16:28] "GET /static/script.js HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:16:32] "GET /api/stock?symbol=RELIANCE.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:11] "GET /api/stock?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:34] "GET /api/stock?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:35] "GET /api/stock?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 09:17:35] "GET /api/export?symbol=TCS.NS&period=1y HTTP/1.1" 200 -
zsh
Python
Tip: Use /create-agent to scaffold a custom agent for your workflow.
index.html
Describe what to ...
Local
Explorer
Open Editors
index.html templates
Stock_M...
static
script.js
style.css
templates
index.html
venv
bin
activate
activate.csh
activate.fish
Activate.ps1
f2py
flask
~/Downloads/Stock_Market_Data_Pipeline_Enhanced/venv/bin/flask
normalizer
numpy-config
pip
pip3
pip3.9
pwiz
pwmigrate
python
python3
python3.9
sample
websockets
include
lib
pyvenv.cfg
.gitignore
Outline
Timeline
Ln 1, Col 1
Spaces: 2
UTF-8
LF
{} HTML

OUTPUT AND RESULTS

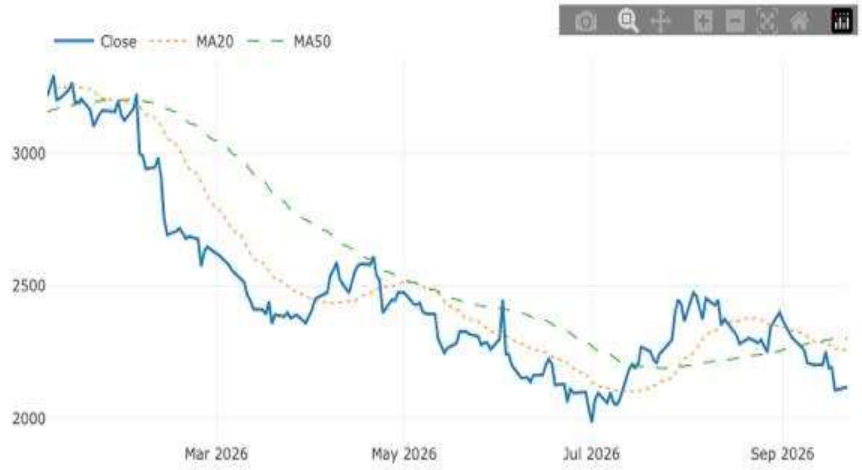


MF MarketFlow
DATA PIPELINE

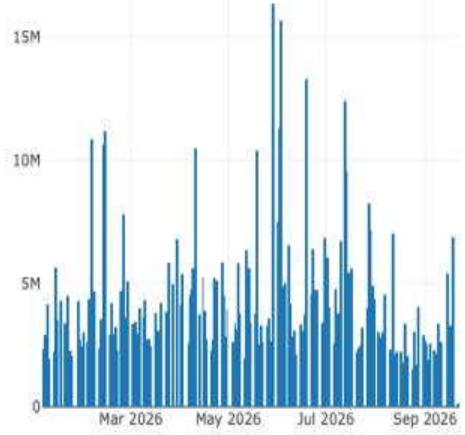
WORKSPACE
- Dashboard
- Pipeline
- Dataset
- Insights

Data source online
Yahoo Finance + API

ANALYSIS
Price & Moving Averages



VOLUME
Trading Activity



PIPELINE MONITOR
Data Quality & Processing

Updated 2026-09-22 09:17:35

01	→	02	→	03	→	04	→	05
COLLECT		CLEAN		TRANSFORM		ANALYZE		VISUALIZE
API / dataset		Nulls & duplicates		Indicators		Patterns		Dashboard

RAW RECORDS	CLEAN RECORDS	MISSING VALUES	DUPLICATES
251	251	0	0

DERIVED INSIGHTS
Pipeline Snapshot

Latest return	+0.57%

ACTIONS
Export & Explore

↓ Download processed CSV

PIPELINE MONITOR

Updated 2026-09-22 09:17:35

Data Quality & Processing

01	02	03	04	05
COLLECT	CLEAN	TRANSFORM	ANALYZE	VISUALIZE
API / dataset	Nulls & duplicates	Indicators	Patterns	Dashboard

RAW RECORDS	CLEAN RECORDS	MISSING VALUES	DUPLICATES
251	251	0	0

DERIVED INSIGHTS

Pipeline Snapshot

Latest return	+0.57%
20-day moving average	2,254.4
50-day moving average	2,299.18
Data status	CLEAN

Indicators are descriptive outputs from the processed dataset; they are not investment recommendations.

ACTIONS

Export & Explore

Download processed CSV

Refresh pipeline

Source	Yahoo Finance via yfinance
Processing	Python + Pandas + NumPy

PROCESSED DATA

TCS.NS

Latest Records

Date	Open	High	Low	Close	Volume	Return %	MA20
2026-09-22	2,135.1	2,135.2	2,116	**2,117**	78,885	0.57%	2,254.4
2026-09-18	2,177.2	2,177.3	2,101.2	**2,105**	68,75,428	-3.88%	2,262.76
2026-09-17	2,171	2,263.6	2,162.5	**2,190**	32,95,917	0.05%	2,272.61

MF **MarketFlow**
DATA PIPELINE

WORKSPACE

- Dashboard
- Pipeline
- Dataset
- Insights

Data source online
Yahoo Finance • API

20-day moving average	2,254.4
50-day moving average	2,299.18
Data status	CLEAN

Indicators are descriptive outputs from the processed dataset; they are not investment recommendations.

Refresh pipeline

Source	Yahoo Finance via yfinance
Processing	Python + Pandas + NumPy

PROCESSED DATA

TCS.NS

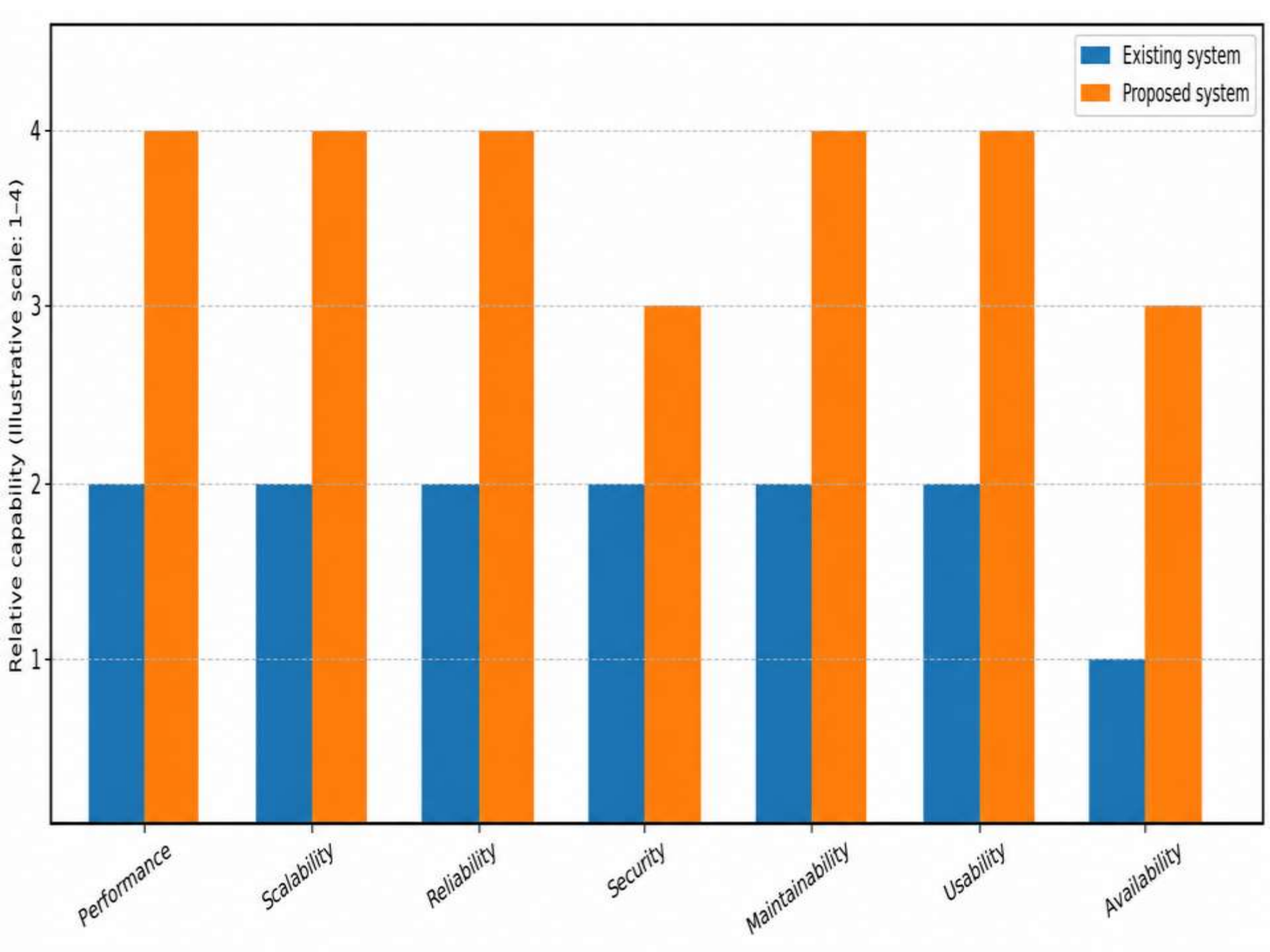
Latest Records

Date	Open	High	Low	Close	Volume	Return %	MA20
2026-09-22	2,135.1	2,135.2	2,116	**2,117**	78,885	0.57%	2,254.4
2026-09-18	2,177.2	2,177.3	2,101.2	**2,105**	68,75,428	-3.88%	2,262.76
2026-09-17	2,171	2,263.6	2,162.5	**2,190**	32,95,917	0.05%	2,272.61
2026-09-16	2,270	2,273.9	2,172	**2,188.8**	27,63,709	-2.76%	2,278.01
2026-09-15	2,275.5	2,323	2,251	**2,251**	54,06,210	2.28%	2,283.02
2026-09-14	2,200.8	2,200.8	2,200.8	**2,200.8**	0	0%	2,284.47
2026-09-11	2,185.5	2,232.6	2,185.5	**2,200.8**	26,34,124	-0.15%	2,290.09
2026-09-10	2,199	2,232	2,190.6	**2,204.1**	24,90,912	-0.18%	2,298.1
2026-09-09	2,237	2,237	2,193.2	**2,208**	33,70,071	-2.11%	2,306.64
2026-09-08	2,270	2,274.5	2,244	**2,255.5**	21,48,114	-0.64%	2,313.72
2026-09-07	2,290.1	2,299.9	2,260.1	**2,270**	22,97,551	-1.48%	2,323.23
2026-09-04	2,330	2,363.9	2,302.4	**2,304**	25,64,322	-0.69%	2,331.02

KLH
(DEEMED TO BE UNIVERSITY)

Comparison

Your System Component	Related Literature / Existing System	What the Existing System Covers	What Your Project Adds / Difference
1. **Data Collection (Python / Stock Market API)**	Stock Market Data Collection and Analysis	Collects stock prices and trading information from financial sources.	Automates collection of stock symbols, dates, Open, High, Low, Close and Volume data using APIs or historical datasets.
2. **Data Cleaning (Pandas)**	Data Preprocessing and Data Quality Management	Handles data preparation before analysis, sometimes through manual processing.	Handles missing, duplicate, inconsistent and incorrect records to improve data quality.
3. **Data Transformation (Pandas / NumPy)**	Financial Data Processing and Feature Engineering	Converts raw financial data into a format suitable for analysis.	Calculates price changes, returns, moving averages and trading-volume patterns.
4. **Data Storage (SQL / Database)**	Financial Data Storage and Management	Stores historical stock market data for later retrieval.	Stores cleaned and transformed stock data in a structured database or file system.
5. **Analytics and Visualization (Python / Matplotlib / Plotly)**	Stock Market Trend Analysis and Visualization	Analyzes stock prices and presents market information through charts.	Combines data analysis with visualizations to identify price movements, trends and trading patterns.
6. **End-to-End Data Pipeline**	Traditional Stock Market Data Analysis Systems	Often requires separate steps for collection, cleaning, analysis and visualization.	Integrates data collection → cleaning → transformation → storage → analysis → visualization into one automated workflow.


Existing system
Proposed system
Relative capability (Illustrative scale: 1–4)
4
3
2
1
Performance
Scalability
Reliability
Security
Maintainability
Usability
Availability

CONCLUSION

- The Stock Market Data Pipeline provides an automated approach to collecting and processing stock market data.
- It improves data quality by handling missing, duplicate, inconsistent, and incorrect records.
- Python, Pandas, and NumPy support efficient data cleaning, transformation, and numerical analysis.
- The system helps analyze stock prices, trading volume, price movements, and market trends.
- Data visualization using Matplotlib or Plotly makes market information easier to understand.
- The integrated pipeline reduces manual effort and supports efficient, data-driven financial analysis.
- The system can be extended to support real-time market monitoring and advanced analytics.

REFERENCES

1. andas Documentation. *Data manipulation, cleaning, and analysis.* https://pandas.pydata.org/docs/
2. NumPy Documentation. *Numerical computing with Python.* https://numpy.org/doc/stable/
3. Matplotlib Documentation. *Data visualization using Python.* https://matplotlib.org/stable/
4. yfinance Documentation. *Downloading historical stock market data.* https://ranaroussi.github.io/yfinance/
5. Python Documentation. *Python programming language and standard library.* https://docs.python.org/3/
6. Project dataset and stock market API documentation used for data collection and analysis.

THANKYOU